Plan 2 — Search Forest

Area-coverage mission · domain `forest-drone-search` · plan kind `area_coverage`
A complete walkthrough of domain.pddl, SearchProblem.pddl and every Python function that runs them.

---

1. What this plan does

Search Forest is the only one of the four missions where **each drone flies its own distinct route**. The operator clicks a **base** point, marks a **search area** as a polygon, and ticks at least two drones. The marked area is then divided into one lane per drone, each drone sweeps its own lane in a back-and-forth lawnmower pattern, and all of them return to the base.

It is also the only plan that **generates** its waypoint graph from nothing. Travell, V-Formation and Grid-Formation all relocate an existing graph onto new points; here the number of waypoints a mission needs depends on how large the marked area is relative to each drone's coverage swath, so the template carries no waypoints at all. Every waypoint object and every route fact is injected fresh on each run.

1.1 The pipeline, end to end

The Search planning pipeline

```
GUI: operator picks a base point, marks a search-area polygon,
     and ticks N drones (N >= 2)
                  |
                  v
services/plan_service.py   PlanService.run_area_async(...)
     -> _Worker.run_area(...)  [on a background QThread]
     -> _Worker._run_area(...)
                  |
                  +--> engine/search_problem.py  expand_search_drones(...)
                  |       the 2-drone/2-lane template -> N drones, N lanes
                  |
                  +--> engine/search_problem.py  retarget_search_problem(...)
                  |       |
                  |       +-- plan_search_lanes(...)  the real geometry:
                  |       |     hull -> min-area rectangle -> equal-area
                  |       |     lane split -> clipped lawnmower sweeps
                  |       |
                  |       +-- _coarsen_route(...)  a short skeleton for ENHSP
                  |       +-- inject waypoints, routes, separations, battery
                  |
                  |       returns (concrete_text, fine_routes)
                  v
     plans/Search/runs/<timestamp>/problem.pddl
                  |
                  v
engine/pddl_planner.py  run_enhsp(domain, problem,
                                  -s gbfs -h hadd -ha true)
     -> parse_plan(stdout) -> list[PlanStep]
     -> extract_multi_route(steps, drone_names)   [a sanity check only]
                  |
                  v
PlanRunResult(per_drone_waypoints = the FINE routes, not ENHSP's)
```

One feature of this pipeline has no counterpart in the other three plans and must be understood before anything else makes sense: **ENHSP's plan is not what the drones fly**. The geometry layer produces the full lawnmower routes; ENHSP is handed a deliberately coarsened skeleton of them and its job is only to confirm that a valid, separation-respecting, battery-feasible schedule exists. Section 4.4 explains why.

2. domain.pddl explained

`plans/Search/domain.pddl` declares the vocabulary of an area-coverage mission. Compared with Travell it adds three genuinely new ideas: **areas** that can be covered, **assignment** of a drone to an area, and **separation** between drones.

2.1 Header, requirements and types

domain.pddl - lines 1-17

```
(define (domain forest-drone-search)

    (:requirements
        :strips :typing :numeric-fluents
        :negative-preconditions :conditional-effects)

    (:types
        drone
        location
        forest-area
        controller
    ))
```

The domain name `forest-drone-search` is what `plan_kind` maps to `area_coverage` in `_DOMAIN_KIND`, which is what routes the GUI's clicks to `run_area_async` rather than `run_async`. This is the only plan reached through that second entry point.

`forest-area` is the new type. It is an **abstract** object, not a place: a drone is never `at` a forest-area. It represents a share of ground to be covered, linked to actual locations by the `area-location` predicate below. That indirection is what lets one lane be declared covered by visiting a single representative waypoint.

2.2 Predicates

Group A - areas, assignment and connectivity

```
;; Drone position
(at ?d - drone ?l - location)

;; Forest areas and their coverage
(forest ?f - forest-area)
(area-location ?a - forest-area ?l - location)
(area-covered ?a - forest-area)

;; Drone-to-area assignment
(assigned ?d - drone ?a - forest-area)

;; Location connectivity
(connected ?from - location ?to - location)
(safe-route ?from - location ?to - location)
(is-base ?l - location)
```

`(assigned ?d ?a)` is the pairing that makes the mission parallel: drone1 owns `forest-lane1`, drone2 owns `forest-lane2`, and so on. Both `start-search` and `cover-area` require it, so no drone can search or claim credit for a lane that is not its own.

`(area-location ?a ?l)` ties an abstract lane to a concrete waypoint. `retarget_search_problem` writes exactly **one** of these per lane, pointing at that lane's first waypoint - so `cover-area` becomes applicable as soon as the drone reaches the start of its lane.

`is-base` marks the single shared launch and recovery point. Unlike Travell there is no `source`/`destination` pair: every drone starts and finishes at the same place.

Group B - drone state and mission state

```
;; Drone states
(flying ?d - drone)
(hovering ?d - drone)
(searching ?d - drone)
(search-completed ?d - drone)
(returning ?d - drone)

;; Battery status
(battery-insufficient ?d - drone)
(battery-checked ?d - drone)

;; Mission status
(forest-search-completed)
(mission-completed ?d - drone)
```

`searching` is set by `start-search` and cleared by `cover-area`, so it marks the interval during which a drone is actively sweeping its lane.

`(forest-search-completed)` is a **nullary** predicate - it takes no arguments and describes the mission as a whole rather than any one drone. It is the gate on `return-to-base`, which is how the domain enforces that nobody goes home until the search as a whole is done.

Group C - separation [the drone-to-drone group]

```
;; Collision avoidance
(safe-separation ?d1 - drone ?d2 - drone)
(separation-ok ?d - drone)
(collision-warning ?d1 - drone ?d2 - drone)
```

This group is the whole reason Search needs two drones minimum, and it is the closest any of the four domains comes to modelling drones reacting to each other. It is discussed in full in chapter 5.

Note the two shapes. `safe-separation` is **pairwise** - it records that a specific pair has been checked against each other. `separation-ok` is **per-drone** - it is the flag `move-search` actually reads. `check-drone-separation` sets all three facts at once: the pair, and the per-drone flag for each of the two drones.

`collision-warning` is declared but never set or read by any action - a diagnostic slot that was never wired up.

Group D - health and the controller channel

```
;; System health
(gps-ok ?d - drone) (communication-ok ?d - drone) (drone-healthy ?d - drone)

;; Failure states
(gps-lost ?d - drone) (communication-lost ?d - drone) (health-failure ?d - drone)

;; Controller commands
(controller-notified ?d - drone)
(controller-return ?d - drone)
(controller-hover ?d - drone)
```

Identical in shape to Travell's, with one notable difference: this domain declares the three controller predicates but has **no actions that read them**. There is no `notify-ground-controller`, no `controller-order-return`, no `sacrifice-drone` here. The Search domain detects faults but has no modelled response to them - the vocabulary was carried over from the Travell domain without the matching action set.

2.3 Functions

domain.pddl - lines 79-111

```
(:functions
    (battery ?d - drone) (max-battery ?d - drone)
    (minimum-return-battery ?d - drone)
    (energy-required ?from - location ?to - location)
    (distance ?from - location ?to - location)
    (drone-width ?d - drone)
    (minimum-drone-separation ?d1 - drone ?d2 - drone)
    (forest-length ?f - forest-area) (forest-width ?f - forest-area)
    (coverage-width ?d - drone)
    (health ?d - drone) (minimum-health ?d - drone)
    (hover-energy ?d - drone)
    (latitude ?l - location) (longitude ?l - location))
```

Two of these are unusually important because the **Python side reads them out of the file**, rather than the planner using them.

`coverage-width` is the drone's real sensor swath. `_read_coverage_width_m` pulls it out of the problem text and uses it to space the lawnmower passes, so the generated sweep always matches whatever the PDDL claims about the aircraft rather than being kept in sync by hand. No action reads it.

`minimum-drone-separation` is the pairwise floor that `check-drone-separation` compares against - the only genuinely pairwise numeric fluent in any of the four domains.

`forest-length` and `forest-width` are written by `retarget_search_problem` but read by nothing; they are informational. `drone-width` and `hover-energy` are likewise unused.

2.4 The 13 actions

Phase 1 - the two checks

ACTION 1 of 13 - check-battery

```
(:action check-battery
    :parameters (?d - drone ?from - location ?to - location)
    :precondition (and (at ?d ?from)
                       (connected ?from ?to)
                       (> (battery ?d) (energy-required ?from ?to)))
    :effect (battery-checked ?d))
```

Certifies that the drone can afford the next hop. Note the difference from Travell: there is no `check-distance` gate ahead of it, so the check chain here is one step shorter - battery check, then move. As in Travell, the flag is per-drone rather than per-leg, which is why `move-search` repeats the numeric test itself.

ACTION 2 of 13 - notify-insufficient-battery

```
(:action notify-insufficient-battery
    :parameters (?d - drone ?from - location ?to - location)
    :precondition (and (at ?d ?from) (connected ?from ?to)
                       (<= (battery ?d) (energy-required ?from ?to)))
    :effect (battery-insufficient ?d))
```

The inverted mirror of `check-battery`, raising a diagnostic flag when a hop is unaffordable. Nothing depends on that flag, so it never appears in a successful plan. Its practical significance is that `retarget_search_problem` works hard to make sure it is never **needed** - see the battery scaling in section 4.3.

ACTION 3 of 13 - check-drone-separation [the coordination action]

```
(:action check-drone-separation
    :parameters (?d1 - drone ?d2 - drone ?l1 - location ?l2 - location)
    :precondition (and (at ?d1 ?l1)
                       (at ?d2 ?l2)
                       (not (= ?d1 ?d2))
                       (>= (distance ?l1 ?l2)
                           (minimum-drone-separation ?d1 ?d2)))
    :effect (and (safe-separation ?d1 ?d2)
                 (separation-ok ?d1)
                 (separation-ok ?d2)))
```

The one action in the domain whose preconditions span **two drones**. It asks: given where these two drones currently are, is the distance between those positions at least the minimum separation required for this pair? If so, both are cleared to move.

`(not (= ?d1 ?d2))` prevents a drone trivially clearing itself by being compared against its own position, which would make the check vacuous.

The effect clears **both** drones at once. This is what makes the chain in chapter 5 work: each check clears two drones, so a line of neighbour checks propagates clearance across the whole swarm without ever needing every pair to be compared.

This action is also why `MIN_AREA_COVERAGE_DRONES = 2`. With a single drone there is no second drone to bind `?d2` to, so `separation-ok` could never be re-established after the first move, and the mission would be unsolvable from the second hop onward.

Phase 2 - searching

ACTION 4 of 13 - start-search

```
(:action start-search
    :parameters (?d - drone ?a - forest-area ?l - location)
    :precondition (and (at ?d ?l) (forest ?a) (area-location ?a ?l)
                       (assigned ?d ?a)
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d)
                       (not (area-covered ?a)))
    :effect (and (searching ?d) (flying ?d)))
```

Begins a sweep. The drone must be standing at a location that belongs to its **own** assigned, not-yet-covered area, and must be fully healthy. The `(not (area-covered ?a))` precondition stops a drone from restarting a search that is already finished.

ACTION 5 of 13 - cover-area

```
(:action cover-area
    :parameters (?d - drone ?a - forest-area ?l - location)
    :precondition (and (at ?d ?l) (forest ?a) (area-location ?a ?l)
                       (assigned ?d ?a) (searching ?d)
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d)
                       (not (area-covered ?a)))
    :effect (and (area-covered ?a)
                 (search-completed ?d)
                 (not (searching ?d))))
```

Declares a lane searched. It is the only producer of `area-covered`, which `complete-forest-search` and the expanded goal both demand.

This is the domain's central **abstraction**, and the point where the PDDL model and the flown reality deliberately diverge. In PDDL, covering an entire lane is a single atomic action taken at one representative waypoint. In reality the drone flies the full lawnmower sweep `plan_search_lanes` produced. The planner is not asked to reason about coverage geometry at all - that has already been solved exactly, in Python, before ENHSP is invoked.

ACTION 6 of 13 - move-search [the central action]

```
(:action move-search
    :parameters (?d - drone ?from - location ?to - location)
    :precondition (and (at ?d ?from) (connected ?from ?to) (safe-route ?from ?to)
                       (separation-ok ?d)
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d)
                       (battery-checked ?d)
                       (> (battery ?d) (energy-required ?from ?to)))
    :effect (and (at ?d ?to) (not (at ?d ?from))
                 (flying ?d)
                 (decrease (battery ?d) (energy-required ?from ?to))
                 (not (battery-checked ?d))
                 (not (separation-ok ?d))))
```

The movement action, and the Search domain's equivalent of Travell's `travel`. `extract_multi_route` looks for exactly this action name, plus `return-to-base`.

The final two effects are what make the mission a genuine coordination problem. Each move **clears both** `battery-checked` and `separation-ok`, so before every single hop a drone must re-certify its battery **and** be re-cleared against another drone. Separation is not established once at the start - it is re-established, hop by hop, for the entire mission.

That repeated clearing is why a Search plan is so much longer than a Travell plan of comparable length, and why the number of separation-coupled lanes drives planning difficulty so sharply.

ACTION 7 of 13 - complete-forest-search

```
(:action complete-forest-search
    :parameters (?a1 - forest-area ?a2 - forest-area)
    :precondition (and (area-covered ?a1)
                       (area-covered ?a2)
                       (not (= ?a1 ?a2)))
    :effect (forest-search-completed))
```

Declares the mission-wide search finished once **two distinct** areas are covered.

Two is hard-coded here, which is a real limitation: with five lanes, covering any two of them would satisfy this action and unlock everyone's return home. The domain cannot express 'all areas' without a universal quantifier it does not declare.

`expand_search_drones` works around this in the **problem** file rather than the domain: when it grows the template past two drones it appends an explicit `(area-covered forest-laneN)` conjunct to the goal for every lane. The goal, not this action, is what actually forces complete coverage.

Phase 3 - going home

ACTION 8 of 13 - return-to-base

```
(:action return-to-base
    :parameters (?d - drone ?current - location ?base - location)
    :precondition (and (at ?d ?current) (is-base ?base)
                       (connected ?current ?base) (safe-route ?current ?base)
                       (separation-ok ?d)
                       (forest-search-completed)
                       (> (battery ?d) (energy-required ?current ?base)))
    :effect (and (at ?d ?base) (not (at ?d ?current))
                 (not (flying ?d)) (returning ?d)
                 (decrease (battery ?d) (energy-required ?current ?base))))
```

The recovery hop. It still demands `separation-ok`, so even the flight home is separation checked. The `forest-search-completed` precondition is the synchronisation point of the whole mission: no drone may head home until the search as a whole is declared finished, which couples every drone's schedule to the others'.

ACTION 9 of 13 - complete-drone-mission

```
(:action complete-drone-mission
    :parameters (?d - drone ?base - location)
    :precondition (and (at ?d ?base) (returning ?d) (forest-search-completed))
    :effect (and (mission-completed ?d) (not (returning ?d))))
```

Converts a safe arrival back at base into mission success for that drone. The goal demands `mission-completed` for every drone, so every successful plan ends with one of these per drone - the Search equivalent of Travell's `reach-destination`.

ACTION 10 of 13 - emergency-low-battery-return

```
(:action emergency-low-battery-return
    :parameters (?d - drone)
    :precondition (and (<= (battery ?d) (minimum-return-battery ?d)))
    :effect (returning ?d))
```

Sets the `returning` flag as soon as a drone's battery falls to its reserve. It is the only action in any of the four domains with **no** positional or state precondition at all - it depends solely on the numeric comparison. Note that it does not move the drone; it only marks intent, and the drone still needs `return-to-base` to actually get home, which in turn still requires `forest-search-completed`. A drone that runs its battery down before the search finishes is therefore stuck - one reason the Python layer scales the battery pool so generously.

Phase 4 - fault detection

ACTIONS 11, 12 and 13 of 13 - the three detectors

```
(:action detect-gps-loss
    :parameters (?d - drone)
    :precondition (and (flying ?d) (not (gps-ok ?d)))
    :effect (and (gps-lost ?d) (not (flying ?d))))

(:action detect-communication-loss
    :parameters (?d - drone)
    :precondition (and (flying ?d) (not (communication-ok ?d)))
    :effect (and (communication-lost ?d) (not (flying ?d))))

(:action detect-health-failure
    :parameters (?d - drone)
    :precondition (and (flying ?d) (< (health ?d) (minimum-health ?d)))
    :effect (and (health-failure ?d) (not (flying ?d))))
```

The same three detectors Travell has, minus the `-warning` flags, which this domain does not declare. Each requires the drone to be airborne, sets its failure fact, and grounds it.

All three are dormant in practice: the problem file asserts all three `-ok` flags and sets health to 100 against a floor of 40, and no action ever removes them. More importantly, unlike Travell there is nothing for them to lead **to** - this domain has no `notify-ground-controller` and no controller orders, so a detected fault is a dead end in the model.

3. SearchProblem.pddl explained

This plan's problem file is named `SearchProblem.pddl`, not `problem.pddl`. That is why `_find_problem_file` has its fallback branch: any single file whose name ends in `problem.pddl`, case-insensitively, is accepted, so a plan can keep a descriptive filename.

> **The most important thing about this template**
>
> It contains no waypoints and no routes whatsoever - no `connected`, no `safe-route`, no `distance`, no `energy-required`, and only one coordinate pair. All of that is generated at run time by `retarget_search_problem`, because how many waypoints a sweep needs depends on the size of the area the operator actually marked.

SearchProblem.pddl - the objects

```
(define (problem forest-search-001)
    (:domain forest-drone-search)

    (:objects
        drone1 - drone
        drone2 - drone
        ground-controller - controller
        forest-lane1 - forest-area
        forest-lane2 - forest-area
        base - location
    ))
```

Two drones, two lanes, one base. Two is the **minimum** rather than a typical value: `check-drone-separation` needs a second drone to bind its `?d2` parameter to, so a one-drone search cannot re-establish `separation-ok` after its first move.

`expand_search_drones` clones `drone2` and `forest-lane2` to reach any larger count - note that it clones the **second** drone, not the first, unlike every other expander in the codebase which clones the first.

`base` is the only location declared. Every waypoint is added later.

SearchProblem.pddl - starting state and assignment

```
;; Both drones start at the base
(at drone1 base)  (at drone2 base)  (is-base base)

;; Assign one search area to each drone
(assigned drone1 forest-lane1)
(assigned drone2 forest-lane2)

(forest forest-lane1)
(forest forest-lane2)

;; Both drones start with normal system status
(gps-ok drone1) (communication-ok drone1) (drone-healthy drone1)
(gps-ok drone2) (communication-ok drone2) (drone-healthy drone2)
```

Both drones begin at the shared base, healthy, each owning one lane. The one-to-one pairing of drone to lane is what makes the mission parallelisable - no two drones are ever assigned the same area, so no two can be credited with covering the same ground.

SearchProblem.pddl - the numeric state

```
;; Initial battery
(= (battery drone1) 100)   (= (max-battery drone1) 100)
(= (battery drone2) 100)   (= (max-battery drone2) 100)
(= (minimum-return-battery drone1) 30)
(= (minimum-return-battery drone2) 30)

;; Drone size and search coverage
(= (drone-width drone1) 50)     (= (drone-width drone2) 50)
(= (coverage-width drone1) 50)  (= (coverage-width drone2) 50)

;; Minimum distance required between the drones
(= (minimum-drone-separation drone1 drone2) 10)
(= (minimum-drone-separation drone2 drone1) 10)

;; Initial drone health
(= (health drone1) 100)  (= (health drone2) 100)
(= (minimum-health drone1) 40) (= (minimum-health drone2) 40)
```

The `coverage-width` of 50 metres is the number the **Python** layer reads, via `_read_coverage_width_m`, to space the lawnmower passes. Change it here and the generated sweep changes with it - the PDDL file is the single source of truth for that figure.

The 10-metre `minimum-drone-separation` is declared symmetrically in both directions, because `check-drone-separation` may bind the pair either way round. `MIN_LANE_WIDTH_M = 20.0` in the Python layer exists to keep adjacent lanes comfortably past this figure even for a very narrow marked area.

The battery values of 100 are **placeholders**. `retarget_search_problem` overwrites all three battery fluents for every drone, scaling them to what the generated route actually costs - see section 4.3.

SearchProblem.pddl - the bootstrap, and the only coordinate

```
;; Allow the first movement from the common base
(separation-ok drone1)
(separation-ok drone2)

;; Base coordinates
(= (latitude base) 13.0827)
(= (longitude base) 80.2707)
```

Those two `separation-ok` facts are a genuine bootstrap and a nice piece of modelling. Both drones start at the **same** base location, so the distance between them is zero, which is below the 10-metre minimum - `check-drone-separation` could never fire, and no drone could ever make its first move. Asserting the flag in the initial state breaks that deadlock, granting each drone one free move to get clear of the base.

The base coordinate is overwritten by `_set_latlon` with the operator's picked point. It is the only geographic fact in the whole template.

SearchProblem.pddl - the goal

```
(:goal
    (and
        (forest-search-completed)
        (mission-completed drone1)
        (mission-completed drone2)
        (at drone1 base)
        (at drone2 base)))
```

Five conjuncts: the search as a whole must be complete, both drones must have completed their missions, and both must be physically back at the base.

For three or more drones `expand_search_drones` rewrites this goal twice over - first cloning the `drone2` conjuncts for every extra drone, then appending an explicit `(area-covered forest-laneN)` for every lane. That second step is the fix for `complete-forest-search` only requiring two covered areas.

4. Every Python function in the Search path

`engine/search_problem.py` is where the real work happens - it is the largest body of original geometry in the project. It borrows nine helpers from `engine/pddl_problem.py` (`_add_init_facts`, `_add_locations`, `_convex_hull`, `_find_matching_paren`, `_from_local`, `_section_span`, `_set_latlon`, `_to_local`, `haversine_m`) and builds an entire mission on top of them.

4.1 The tuning constants

The six module constants

```
DEFAULT_ENERGY_RATE = 0.0125       # energy units per metre
MIN_LANE_WIDTH_M = 20.0            # floor on each lane's width
BATTERY_SAFETY_MARGIN = 1.6        # headroom over the raw route cost
DEFAULT_MAX_BATTERY = 100.0
RETURN_RESERVE_FRACTION = 0.30     # same proportion as the template's 30/100
PDDL_WAYPOINTS_PER_LANE = 5        # turn-points handed to ENHSP per lane
```

`MIN_LANE_WIDTH_M` pads a thin marked area so that its total width is at least 20 m per lane, keeping adjacent sweeps comfortably beyond the domain's 10 m minimum separation. The polygon itself is not changed - per-pass clipping simply returns nothing for a padded sliver that falls outside the real shape, and that pass is skipped.

`BATTERY_SAFETY_MARGIN` and its two companions drive the battery rescaling in section 4.3. The docstring is careful to say this is **not** a real battery model - real physical feasibility is screened separately in milliamp-hour terms by `services/local_flight.plan_route_flights_by_drone`. The margin exists only so that a large but genuinely flyable area is not reported unsolvable because of the template's hardcoded 100.

`PDDL_WAYPOINTS_PER_LANE = 5` is the coarsening budget, and the single most consequential number in this module - see section 4.4.

4.2 The geometry, in the order it runs

_read_coverage_width_m(problem_text)

```
_COVERAGE_WIDTH_FACT = re.compile(r"\(=\s*\(coverage-width\s+\S+\)\s*([-\d.eE]+)\s*\)")

def _read_coverage_width_m(problem_text: str, default: float = 50.0) -> float:
    match = _COVERAGE_WIDTH_FACT.search(problem_text)
    return float(match.group(1)) if match else default
```

Reads the drone's coverage swath back out of the PDDL file. This small function is what keeps the generated geometry honest: the sweep spacing is derived from whatever the problem file claims rather than from a Python constant that would have to be kept in sync with it by hand. It takes the first match, assuming all drones share a swath.

_min_area_rect(hull) - rotating calipers

```
def _min_area_rect(hull):
    n = len(hull)
    best = None
    for i in range(n):
        ax, ay = hull[i]; bx, by = hull[(i + 1) % n]
        dx, dy = bx - ax, by - ay
        length = math.hypot(dx, dy)
        if length < 1e-9: continue
        u_dir = (dx / length, dy / length)
        v_dir = (-u_dir[1], u_dir[0])
        us = [p[0]*u_dir[0] + p[1]*u_dir[1] for p in hull]
        vs = [p[0]*v_dir[0] + p[1]*v_dir[1] for p in hull]
        u_min, u_max, v_min, v_max = min(us), max(us), min(vs), max(vs)
        area = (u_max - u_min) * (v_max - v_min)
        if best is None or area < best[0]:
            best = (area, u_dir, v_dir, u_min, u_max, v_min, v_max)
    _, length_dir, width_dir, u_min, u_max, v_min, v_max = best
    if (u_max - u_min) < (v_max - v_min):
        length_dir, width_dir = width_dir, length_dir
        u_min, u_max, v_min, v_max = v_min, v_max, u_min, u_max
    return length_dir, width_dir, u_min, u_max, v_min, v_max
```

Finds the hull's true minimum-area bounding rectangle and returns its two axis directions plus the hull's extent along each. `length_dir` becomes the sweep axis - the direction the drones fly back and forth - and `width_dir` the axis the lanes are split along.

The method relies on a classical theorem: the minimum-area rectangle bounding a convex polygon always has one side flush with one of the polygon's edges. So trying every edge direction as a candidate orientation and keeping the smallest-area result is **exact**, not an approximation.

The docstring explains why the obvious shortcut was rejected: guessing the orientation from the single longest edge can be badly fooled by a trapezoid or a skewed quadrilateral, producing a split whose lanes cross each other.

The final swap enforces `length >= width`, so the drones always sweep along the long axis - fewer, longer passes and fewer turns.

_polygon_area and _clip_below

```
def _polygon_area(polygon) -> float:
    n = len(polygon)
    if n < 3: return 0.0
    total = 0.0
    for i in range(n):
        x1, y1 = polygon[i]; x2, y2 = polygon[(i + 1) % n]
        total += x1 * y2 - x2 * y1
    return abs(total) / 2.0

def _clip_below(polygon, v_threshold):
    n = len(polygon)
    if n < 3: return []
    result = []
    for i in range(n):
        cur = polygon[i]; nxt = polygon[(i + 1) % n]
        cur_in = cur[1] <= v_threshold
        nxt_in = nxt[1] <= v_threshold
        if cur_in: result.append(cur)
        if cur_in != nxt_in:
            dv = nxt[1] - cur[1]
            t = (v_threshold - cur[1]) / dv if abs(dv) > 1e-12 else 0.0
            result.append((cur[0] + t * (nxt[0] - cur[0]), v_threshold))
    return result
```

`_polygon_area` is the shoelace formula - the signed area of a polygon from its vertices, made unsigned so winding order does not matter.

`_clip_below` is one step of the Sutherland-Hodgman clipping algorithm: it intersects the polygon with the half-plane below a horizontal threshold. It walks the edges and keeps every vertex already inside, and wherever an edge **crosses** the threshold it interpolates the exact crossing point and inserts that too.

Together these two answer the question `_equal_area_splits` needs: how much of the marked area lies below a given line?

_equal_area_splits(...) - fair shares, not equal widths

```
def _equal_area_splits(hull_uv, v_min, v_max, n_lanes) -> list[float]:
    total = _polygon_area(hull_uv)
    interior = []
    for i in range(1, n_lanes):
        target = total * i / n_lanes
        lo, hi = v_min, v_max
        for _ in range(40):                       # binary search
            mid = (lo + hi) / 2.0
            if _polygon_area(_clip_below(hull_uv, mid)) < target: lo = mid
            else: hi = mid
        interior.append((lo + hi) / 2.0)

    eps = min(1.0, (v_max - v_min) / (4.0 * n_lanes))
    lower = v_min + eps
    for i, s in enumerate(interior):
        s = max(s, lower)
        s = min(s, v_max - eps * (n_lanes - 1 - i))
        interior[i] = s
        lower = s + eps
    return [v_min, *interior, v_max]
```

Returns the lane boundaries that carve the marked area into strips of **equal area**. Each interior boundary is found by binary search: the threshold below which the polygon holds exactly its share of the total area.

The search is guaranteed to converge because, for a convex polygon, the clipped area is monotonically non-decreasing in the threshold. Forty iterations of bisection over a span of a few kilometres resolves to well under a millimetre - far more precision than the problem needs.

Why equal area rather than equal width. For a rectangle the two are the same, but for a triangle or trapezoid they are very different: equal-width bands would give the drone at the narrow end a strip covering a small fraction of the ground the drone at the wide end must sweep. Equal area gives every drone a genuinely fair share of work.

The `eps` pass afterwards is a degeneracy guard. If a pathological shape collapses two boundaries onto each other, the nudging keeps them strictly ordered and separated, so every lane retains at least a sliver of span and no lane comes out empty.

_u_range_at_v(hull_uv, v)

```
def _u_range_at_v(hull_uv, v) -> Optional[tuple[float, float]]:
    n = len(hull_uv); us = []
    for i in range(n):
        a, b = hull_uv[i], hull_uv[(i + 1) % n]
        a_v, b_v = a[1], b[1]
        if (a_v - v) * (b_v - v) > 1e-9: continue    # both strictly on one side
        if abs(b_v - a_v) < 1e-9:
            if abs(a_v - v) < 1e-9: us.append(a[0]); us.append(b[0])
            continue
        t = (v - a_v) / (b_v - a_v)
        us.append(a[0] + t * (b[0] - a[0]))
    if not us: return None
    return min(us), max(us)
```

Given the height of one sweep pass, returns the interval where that horizontal line actually crosses the marked polygon - or `None` if it misses it entirely.

This is the function that makes the sweeps honest. Without it, every pass would be stretched across the full width of the bounding rectangle, and a pass near a tapering corner of a triangular area would send the drone flying a long way over ground that was never marked. With it, each pass is exactly as long as the real ground at that position.

The sign-product test `(a_v - v) * (b_v - v) > 1e-9` skips edges with both endpoints strictly on the same side of the line. The horizontal-edge branch handles an edge lying exactly along the sweep line, contributing both its endpoints.

_sweep_passes(v_lo, v_hi, coverage_width_m)

```
def _sweep_passes(v_lo, v_hi, coverage_width_m) -> list[float]:
    span = max(v_hi - v_lo, 1e-6)
    count = max(1, math.ceil(span / max(coverage_width_m, 1e-6)))
    step = span / count
    return [v_lo + step * (i + 0.5) for i in range(count)]
```

Places the sweep lines within one lane. It takes the smallest number of passes whose strips each stay within the coverage width, then spaces them **evenly** across the lane rather than packing them at full width from one edge.

The `+ 0.5` centres each pass in its own strip. The combination of `ceil` for the count and an even `span / count` step means the lane is tiled exactly: no gap between strips, and no strip wider than the drone can actually see.

plan_search_lanes(...) - the complete coverage geometry [the key function]

```
def plan_search_lanes(base, corners, coverage_width_m=50.0, n_lanes=2):
    if len(corners) < 3:
        raise ValueError("a forest search area needs at least 3 corners")
    n_lanes = max(1, n_lanes)
    hull = _convex_hull([_to_local(p, base) for p in corners])
    if len(hull) < 3:
        raise ValueError("the marked forest area is degenerate (all corners collinear)")

    length_dir, width_dir, u_min, u_max, v_min, v_max = _min_area_rect(hull)
    hull_uv = [(p[0]*length_dir[0] + p[1]*length_dir[1],
                p[0]*width_dir[0]  + p[1]*width_dir[1]) for p in hull]

    base_u = 0.0                        # base is the local origin
    near_is_min = abs(base_u - u_min) <= abs(base_u - u_max)

    min_total_width = MIN_LANE_WIDTH_M * n_lanes
    if v_max - v_min < min_total_width:
        pad_mid = (v_min + v_max) / 2.0
        v_min, v_max = pad_mid - min_total_width/2.0, pad_mid + min_total_width/2.0

    bounds = _equal_area_splits(hull_uv, v_min, v_max, n_lanes)

    def to_world(u, v):
        x = u*length_dir[0] + v*width_dir[0]
        y = u*length_dir[1] + v*width_dir[1]
        return _from_local((x, y), base)

    routes = {}
    for lane in range(n_lanes):
        v_lo, v_hi = bounds[lane], bounds[lane + 1]
        turns = []
        for i, v in enumerate(_sweep_passes(v_lo, v_hi, coverage_width_m)):
            span = _u_range_at_v(hull_uv, v)
            if span is None: continue
            lo_u, hi_u = span
            near_u, far_u = (lo_u, hi_u) if near_is_min else (hi_u, lo_u)
            u_a, u_b = (near_u, far_u) if i % 2 == 0 else (far_u, near_u)
            turns.append((u_a, v)); turns.append((u_b, v))
        routes[f"drone{lane + 1}"] = [base] + [to_world(u, v) for u, v in turns] + [base]
    return routes
```

Produces the real route every drone flies: a dictionary mapping each drone name to its full `[base, waypoint, ..., base]` sweep. Everything else in the module supports this function.

Step 1 - normalise the shape. The clicked corners are projected into local metres and reduced to their convex hull, which as in Travell fixes the bowtie problem caused by a natural, non-boundary-walking click order.

Step 2 - choose the axes. `_min_area_rect` gives the sweep direction and the split direction, and the hull is re-expressed in that rotated frame as `hull_uv`. From here on the problem is axis-aligned, which is what makes the clipping and splitting simple.

Step 3 - pad a thin area. If the whole area is narrower than 20 m per lane, the span is widened symmetrically about its midpoint. Crucially the **polygon** is untouched, so the padding creates no phantom ground: passes that fall outside the real shape simply get `None` back from `_u_range_at_v` and are skipped.

Step 4 - split and sweep. The lane boundaries come from the equal-area search, and each lane is swept by alternating direction on successive passes - the `i % 2 == 0` test is the boustrophedon, the back-and-forth that makes it a lawnmower pattern rather than a series of disconnected parallel lines.

The `near_is_min` detail. Since the base sits at the local origin, its projection onto the sweep axis is zero, and comparing that against the hull's two extremes says which end of the area is nearer to home. The first pass then starts at the near end, so a drone does not fly the length of the area before beginning work.

Because the lanes come from an equal-area split of one hull they are geometrically disjoint by construction: no two drones can ever fly through, or be credited with, the same ground.

4.3 Building the problem file

_coarsen_route(points, max_interior)

```
def _coarsen_route(points, max_interior) -> list[LatLon]:
    interior = points[1:-1]
    if max_interior <= 0 or len(interior) <= max_interior:
        return list(points)
    if max_interior == 1:
        picks = [interior[0]]
    else:
        step = (len(interior) - 1) / (max_interior - 1)
        picks = [interior[round(i * step)] for i in range(max_interior)]
    return [points[0], *picks, points[-1]]
```

Reduces a full lawnmower sweep to at most `max_interior` turn-points, keeping both `base` endpoints and spreading the picks evenly - always including the first and last interior point, so the skeleton still spans the whole lane.

This is the function that decouples what ENHSP reasons about from what the drones fly. A real sweep of a large area can have dozens of turn-points; handing all of them to a planner that must also interleave separation checks between every pair of adjacent lanes produces a search space that does not finish. Five points per lane keeps the plan's **shape** - go to the lane, search it, come home - while making it solvable in seconds.

retarget_search_problem - part 1: geometry and waypoint objects

```
def retarget_search_problem(problem_text, base, corners,
                            coverage_width_m=None, n_lanes=None,
                            pddl_waypoints_per_lane=PDDL_WAYPOINTS_PER_LANE):
    if coverage_width_m is None:
        coverage_width_m = _read_coverage_width_m(problem_text)
    if n_lanes is None:
        n_lanes = max(2, len(re.findall(r"\(assigned\s+drone\d+\s", problem_text)))
    fine_routes = plan_search_lanes(base, corners, coverage_width_m, n_lanes)
    routes = {drone: _coarsen_route(points, pddl_waypoints_per_lane)
              for drone, points in fine_routes.items()}
    drone_ids = list(routes)

    names = {}
    for drone, points in routes.items():
        interior = points[1:-1]
        names[drone] = [f"{drone}-wp{i+1}" for i in range(len(interior))]
    result = _add_locations(problem_text,
                            [n for dn in names.values() for n in dn])
```

The module's main entry point. It returns a **pair**: the concrete problem text, and the fine routes the drones will actually fly.

Both optional parameters default to reading the answer out of the problem text itself. `n_lanes` is inferred by counting `(assigned droneN ...)` facts, so a caller that has already run `expand_search_drones` does not have to state the drone count twice - and the two can never disagree.

The fine routes are computed first, then coarsened. Both are kept: the coarse one becomes PDDL waypoint objects named `drone1-wp1`, `drone1-wp2`, and so on; the fine one is returned to the caller and is what flies.

retarget_search_problem - part 2: the per-lane route chain

```
    facts = []
    for lane_index, (drone, points) in enumerate(routes.items(), start=1):
        drone_names = names[drone]
        chain_names = ["base"] + drone_names + ["base"]
        chain_points = points

        for name, point in zip(drone_names, points[1:-1]):
            facts.append(f"\n        (= (latitude {name}) {point.lat:.7f})")
            facts.append(f"\n        (= (longitude {name}) {point.lon:.7f})")

        for a_name, a_pt, b_name, b_pt in zip(chain_names, chain_points,
                                              chain_names[1:], chain_points[1:]):
            distance_m = haversine_m(a_pt, b_pt)
            energy = distance_m * DEFAULT_ENERGY_RATE
            facts.append(f"\n        (connected {a_name} {b_name})")
            facts.append(f"\n        (safe-route {a_name} {b_name})")
            facts.append(f"\n        (= (distance {a_name} {b_name}) {distance_m:.2f})")
            facts.append(f"\n        (= (energy-required {a_name} {b_name}) {energy:.2f})")

        if drone_names:
            facts.append(
                f"\n        (area-location forest-lane{lane_index} {drone_names[0]})")
```

Builds each lane's waypoint chain from scratch: coordinates for every waypoint, then `connected`, `safe-route`, `distance` and `energy-required` for every consecutive pair from the base out through the lane and back to the base.

Note that unlike Travell's `_set_detour_route`, these legs are written in **one direction only**, following the sweep order. A Search drone never needs to fly a leg backwards, and omitting the reverse facts halves the number of `move-search` actions ENHSP has to ground.

The single `area-location` fact per lane points at that lane's **first** waypoint. That is what makes `start-search` and `cover-area` applicable the moment the drone arrives at the beginning of its lane, which is why coverage costs only one action in the plan.

retarget_search_problem - part 3: cross-lane separation distances

```
    for a, b in zip(drone_ids, drone_ids[1:]):
        na, nb = len(names[a]), len(names[b])
        common = min(na, nb)
        for i in range(common):
            a_name, b_name = names[a][i], names[b][i]
            distance_m = haversine_m(routes[a][i+1], routes[b][i+1])
            facts.append(f"\n        (= (distance {a_name} {b_name}) {distance_m:.2f})")
            facts.append(f"\n        (= (distance {b_name} {a_name}) {distance_m:.2f})")
        if na != nb:
            longer = a if na > nb else b
            for i in range(common, len(names[longer])):
                name = names[longer][i]
                distance_m = haversine_m(routes[longer][i+1], base)
                facts.append(f"\n        (= (distance {name} base) {distance_m:.2f})")
                facts.append(f"\n        (= (distance base {name}) {distance_m:.2f})")
```

Writes the distances `check-drone-separation` compares against. This block is the numeric backing for the entire coordination story in chapter 5, and it repays careful reading.

Only adjacent lanes are paired. The loop walks `zip(drone_ids, drone_ids[1:])`, so lane 1 is paired with lane 2, lane 2 with lane 3, and so on - never lane 1 with lane 5. This is sufficient because clearance propagates along the chain, and it keeps the number of facts linear in the drone count rather than quadratic.

Waypoints are paired by index. Waypoint `i` of one lane is measured against waypoint `i` of the neighbouring lane - the two drones' positions at corresponding points in their sweeps.

The ragged-lane fallback. Because the split is by area rather than width, neighbouring lanes over a lopsided shape can have different numbers of passes. Index `i` therefore only lines up as far as the shorter lane. Beyond that, the longer lane's remaining waypoints are measured against `base` instead. The comment justifies this precisely: by then the shorter lane's drone has necessarily finished its own waypoints and is free to be home, nothing in the domain ties one drone's progress to another's, and `base` - being outside the marked area - is always a safe distance away.

retarget_search_problem - part 4: dimensions, base, and the battery rescale

```
    hull = _convex_hull([_to_local(p, base) for p in corners])
    _, _, u_min, u_max, v_min, v_max = _min_area_rect(hull)
    lane_width_m = max(v_max - v_min, MIN_LANE_WIDTH_M * n_lanes) / n_lanes
    lane_length_m = abs(u_max - u_min)
    for lane_index in range(1, n_lanes + 1):
        facts.append(f"\n        (= (forest-width forest-lane{lane_index}) {lane_width_m:.2f})")
        facts.append(f"\n        (= (forest-length forest-lane{lane_index}) {lane_length_m:.2f})")

    result = _add_init_facts(result, "".join(facts))
    result = _set_latlon(result, "base", base)

    route_energy = {
        drone: sum(haversine_m(a, b) for a, b in zip(points, points[1:]))
               * DEFAULT_ENERGY_RATE
        for drone, points in fine_routes.items()}
    max_battery = max(DEFAULT_MAX_BATTERY,
                      max(route_energy.values()) * BATTERY_SAFETY_MARGIN)
    return_reserve = max_battery * RETURN_RESERVE_FRACTION
    for drone in fine_routes:
        result = _set_unary_fact(result, "battery", drone, max_battery)
        result = _set_unary_fact(result, "max-battery", drone, max_battery)
        result = _set_unary_fact(result, "minimum-return-battery", drone, return_reserve)
    return result, fine_routes
```

The closing block. The `forest-width`/`forest-length` facts are informational only, as noted in section 2.3 - nothing reads them.

The battery rescale is the important part, and it is the reason this plan has **zero** unsolvable runs on record out of 69. The energy each drone's route really costs is summed leg by leg, the largest is multiplied by the 1.6 safety margin, and every drone's battery, maximum battery and return reserve are rewritten to match - with `DEFAULT_MAX_BATTERY` as a floor so a small area is not given a battery smaller than the template's.

The subtlety worth noting: the scale is computed from `fine_routes`, the **full** sweep, not from the coarsened skeleton ENHSP sees. Sizing the battery to the short skeleton would make the feasibility check meaningless - it would certify a mission the drones could not actually fly. Using the fine route keeps the check honest even though the planner reasons over less.

4.4 Why ENHSP is given a skeleton

The central design decision of this plan

The drones fly the fine lawnmower routes; ENHSP only ever sees at most five turn-points per lane. The module's own comment states the reason plainly: with greedy best-first search and helpful actions the coarsened problem solves in a few seconds even for five or six separation-coupled lanes, whereas the default weighted-A-star search on the full routes does not return at all. Plan **optimality** does not matter here, because the drones fly the fixed geometric routes regardless of the order ENHSP happens to sequence the hops in. The planner's role is reduced to answering one question: does a valid, separation-respecting, battery-feasible schedule exist?

4.5 Swarm expansion

_clone_search_lines(text, span, extra, skip=None)

```
_SEPARATION_LINE = re.compile(r"minimum-drone-separation")

def _clone_search_lines(text, span, extra, *, skip=None) -> str:
    _open, close_pos = span
    clones = []
    for line in text[_open:close_pos].splitlines():
        if skip is not None and skip.search(line): continue
        if re.search(r"\bdrone2\b", line) or re.search(r"\bforest-lane2\b", line):
            for k in extra:
                new_line = re.sub(r"\bdrone2\b", f"drone{k}", line)
                new_line = re.sub(r"\bforest-lane2\b", f"forest-lane{k}", new_line)
                clones.append(new_line)
    if not clones: return text
    return (f"{text[:close_pos].rstrip()}\n" + "\n".join(clones)
            + f"\n    {text[close_pos:]}")
```

The Search plan's own cloning helper, parallel to `_clone_named_lines` in `engine/pddl_problem.py` but differing in two ways.

First, it substitutes **two** tokens at once - `drone2` and `forest-lane2` - because a drone and its lane must stay paired. A line such as `(assigned drone2 forest-lane2)` has to become `(assigned drone5 forest-lane5)`, not some mismatched pair.

Second, it takes a `skip` pattern. That is used to exclude the `minimum-drone-separation` facts from cloning, because those are pairwise and cannot be produced by substituting a single name - they are stripped and rebuilt from scratch in the next step.

Note that it clones `drone2`, the **second** drone, rather than the first. The template's first drone carries lane-1-specific facts, so the second is the better prototype for further lanes.

expand_search_drones(problem_text, count)

```
def expand_search_drones(problem_text: str, count: int) -> str:
    if count <= 2: return problem_text
    drones = [f"drone{i}" for i in range(1, count + 1)]
    lanes = [f"forest-lane{i}" for i in range(1, count + 1)]
    extra = range(3, count + 1)

    sep_match = re.search(
        r"\(=\s*\(minimum-drone-separation\s+\S+\s+\S+\)\s*([-\d.eE]+)\s*\)", problem_text)
    sep_value = sep_match.group(1) if sep_match else "10"

    # 1. widen the (:objects ...) declarations  (both drone and forest-area)
    obj_open, obj_close = _section_span(problem_text, "objects")
    block = problem_text[obj_open:obj_close]
    block = re.sub(r"\bdrone2\b(\s*-\s*drone\b)", ..., block, count=1)
    block = re.sub(r"\bforest-lane2\b(\s*-\s*forest-area\b)", ..., block, count=1)
    problem_text = problem_text[:obj_open] + block + problem_text[obj_close:]

    # 2. clone every drone2 / forest-lane2 (:init ...) line, minus separations
    problem_text = _clone_search_lines(
        problem_text, _section_span(problem_text, "init"), extra, skip=_SEPARATION_LINE)

    # 3. rebuild minimum-drone-separation for ADJACENT lane pairs only
    problem_text = re.sub(
        r"[ \t]*\(=\s*\(minimum-drone-separation\b[^\n]*\)[ \t]*\n?", "", problem_text)
    _init_open, init_close = _section_span(problem_text, "init")
    sep_facts = "".join(
        f"\n        (= (minimum-drone-separation {a} {b}) {sep_value})"
        f"\n        (= (minimum-drone-separation {b} {a}) {sep_value})"
        for a, b in zip(drones, drones[1:]))
    problem_text = f"{problem_text[:init_close].rstrip()}{sep_facts}\n    {problem_text[init_close:]}"

    # 4. goal: clone the drone2 conjuncts, then force EVERY lane covered
    ... # locate the goal's (and ...) span, clone into it
    covered = "".join(f"\n            (area-covered {lane})" for lane in lanes)
    problem_text = f"{problem_text[:close_at].rstrip()}{covered}\n    {problem_text[close_at:]}"
    return problem_text
```

Grows the 2-drone, 2-lane template to any larger count, in four steps.

Step 1 widens two object declarations rather than one, since drones and lanes grow together.

Step 2 clones the per-drone initial facts, skipping the pairwise separation lines.

Step 3 is the performance-critical one, and its comment says why. Every existing separation fact is deleted and rebuilt for **adjacent pairs only** - drone1 with drone2, drone2 with drone3, and so on. If every pair were declared, ENHSP would ground a `check-drone-separation` action for every unrelated pair of drones at every pair of positions, which for four or more drones bloats the search badly. Restricting the fact to neighbouring lanes restricts the grounding, and a chain of neighbour checks still makes every drone `separation-ok`.

Step 4 fixes the two-area limitation of `complete-forest-search` discussed in section 2.4. After cloning the per-drone goal conjuncts, an explicit `(area-covered forest-laneN)` is appended for **every** lane, so the goal itself demands complete coverage even though the action only requires two areas.

4.6 The orchestration side

_area_drone_names(count)

```
MIN_AREA_COVERAGE_DRONES = 2

def _area_drone_names(count: int) -> list[str]:
    return [f"drone{i}" for i in range(1, max(MIN_AREA_COVERAGE_DRONES, count) + 1)]
```

Produces the drone object names for a search, matching exactly what `expand_search_drones` writes into the file. The `max` against the minimum of two silently raises a one-drone request to two, because - as section 2.4 explained - a single drone can never satisfy `check-drone-separation` and the mission would be unsolvable.

_Worker.run_area(...) - the second entry point

```
@Slot(str, object, object, int)
def run_area(self, plan_name, base, corners, drone_count=2) -> None:
    try:
        result = self._run_area(plan_name, base, corners, drone_count)
    except PlanNotFound as exc:
        self._save_stdout(plan_name, exc.stdout)
        self.failed.emit(str(exc))
    except PlannerError as exc:
        self.failed.emit(str(exc))
    except (OSError, ValueError) as exc:
        self.failed.emit(f"Could not plan the search area: {exc}")
    else:
        self.finished.emit(result)
```

The Search plan's own thread-side slot, separate from `run` because it is driven by different clicks: a base point and a polygon, rather than a source, a destination and an optional restriction.

The error handling parallels `run`'s, with `OSError` and `ValueError` merged into one branch under a message that names the search area. That `ValueError` branch is what catches the two guards inside `plan_search_lanes` - fewer than three corners, or a degenerate all-collinear area - and turns them into a readable message rather than a traceback.

_Worker._run_area(...) - the Search run, start to finish

```
def _run_area(self, plan_name, base, corners, drone_count=2) -> PlanRunResult:
    n_lanes = max(MIN_AREA_COVERAGE_DRONES, drone_count)
    domain_path, template_path = self._plan_files(plan_name)
    template_text = template_path.read_text(encoding="utf-8")
    template_text = expand_search_drones(template_text, n_lanes)
    concrete_text, fine_routes = retarget_search_problem(
        template_text, PddlLatLon(lat=base.lat, lon=base.lon),
        [PddlLatLon(lat=p.lat, lon=p.lon) for p in corners], n_lanes=n_lanes)

    run_dir = self._start_run(plan_name, concrete_text)
    steps, stdout = run_enhsp(
        domain_path, run_dir / "problem.pddl",
        timeout_s=max(45.0, 15.0 * n_lanes),
        extra_args=["-s", "gbfs", "-h", "hadd", "-ha", "true"])
    (run_dir / "plan.txt").write_text(stdout, encoding="utf-8")

    drone_names = _area_drone_names(n_lanes)
    moved = extract_multi_route(steps, drone_names)
    missing = [d for d in drone_names if d not in moved]
    if missing:
        raise PlanNotFound("ENHSP found a plan but it never moves: " + ", ".join(missing))
    per_drone_waypoints = {
        drone: [LatLon(lat=p.lat, lon=p.lon) for p in fine_routes[drone]]
        for drone in drone_names}

    first_drone = drone_names[0]
    return PlanRunResult(plan_name=plan_name, steps=steps,
                         waypoints=per_drone_waypoints[first_drone],
                         location_names=moved[first_drone], run_dir=run_dir,
                         per_drone_waypoints=per_drone_waypoints)
```

Expand before retarget - the opposite order from Travell's `_run`. The reason is that `retarget_search_problem` generates geometry **per lane**, so it must already know how many lanes there are. Travell retargets a fixed graph and then clones drones onto it; here the drone count determines the graph.

The ENHSP invocation is tuned. The timeout scales with the lane count - 15 seconds per lane, floor of 45 - and the search strategy is overridden to greedy best-first (`-s gbfs`) with the additive heuristic (`-h hadd`) and helpful actions (`-ha true`). As section 4.4 explains, the default weighted-A-star search does not return on this problem.

ENHSP's route is a sanity check, not an output. `extract_multi_route` is called only to verify that every drone actually moves in the returned plan; a plan that leaves a drone stationary raises `PlanNotFound` naming the offenders. What goes into `per_drone_waypoints` is `fine_routes` - the geometry layer's full lawnmower sweeps.

The first drone's route is duplicated into `waypoints`. That field exists for display code that only knows about 'the' route, such as a status line showing a distance. Actual execution always reads `per_drone_waypoints`.

extract_multi_route(steps, drones) [engine/pddl_planner.py]

```
def extract_multi_route(steps, drones) -> dict[str, list[str]]:
    routes = {}
    ordered = sorted(steps, key=lambda s: s.index)
    for drone in drones:
        legs = [step.args[1:3] for step in ordered
                if step.action in ("move-search", "return-to-base")
                and step.args and step.args[0] == drone]
        if not legs: continue
        route = [legs[0][0]]
        route.extend(to for _from, to in legs)
        routes[drone] = route
    return routes
```

The per-drone counterpart to Travell's `extract_route`. Instead of one shared route it builds one route per drone, and it recognises **two** movement action names rather than one: `move-search` for the sweep and `return-to-base` for the trip home. Both count as a hop for whichever drone appears in the first argument.

The chaining is identical - the first leg's origin followed by every leg's destination - and a drone with no movement actions is simply absent from the result, which is precisely what `_run_area` tests for when it computes `missing`.

5. How the drones communicate

Search is the first of the four plans in which drones are genuinely **coupled to each other** in the model. It is worth separating three distinct mechanisms.

5.1 Drone to ground: the same single link as Travell

The ground link

```
(communication-ok ?d - drone)      ;; healthy radio link to the ground controller
(communication-lost ?d - drone)    ;; detected as down by detect-communication-loss
```

`communication-ok` is required by `start-search`, `cover-area` and `move-search`, so a drone that has lost its link can neither search nor move. Unlike Travell, however, there is nowhere for a detected loss to go: this domain declares `controller-notified`, `controller-return` and `controller-hover` but has no action that produces or consumes any of them. Fault detection is modelled; fault **response** is not.

5.2 Drone to drone: separation, and how clearance propagates

This is the real coordination mechanism, and the thing that makes Search a multi-drone problem rather than several independent single-drone problems run at once.

The separation protocol

```
  Before EVERY hop, a drone must hold (separation-ok ?d).
  Every move-search CLEARS it again.

  check-drone-separation (?d1 ?d2 ?l1 ?l2)
      requires:  (at ?d1 ?l1), (at ?d2 ?l2), ?d1 != ?d2,
                 distance(?l1, ?l2) >= minimum-drone-separation(?d1, ?d2)
      grants:    (safe-separation ?d1 ?d2)
                 (separation-ok ?d1)      <-- BOTH drones are cleared
                 (separation-ok ?d2)          by one single check


  The declared pairs form a CHAIN, not a full mesh:

      drone1 <---> drone2 <---> drone3 <---> drone4 <---> drone5
         (lane 1)    (lane 2)     (lane 3)     (lane 4)    (lane 5)

  One check per adjacent pair clears every drone in the swarm:
      check(d1,d2) clears d1 and d2
      check(d2,d3) clears d2 and d3
      check(d3,d4) clears d3 and d4   ... and so on

  => N-1 checks for N drones, instead of N(N-1)/2 for a full mesh.
```

Three properties make this work, and each is a deliberate choice.

Clearance is per-hop, not once per mission. `move-search` deletes `separation-ok` in its own effect, so the check must be repeated before every single waypoint. A drone's freedom to move is continuously re-earned against where the other drones currently are.

One check clears two drones. Because `check-drone-separation` grants `separation-ok` to both of its parameters, clearance spreads along the chain. A line of neighbour checks suffices; no drone ever needs to be compared against a drone in a distant lane.

Only neighbouring lanes are declared as pairs. `expand_search_drones` writes `minimum-drone-separation` facts for adjacent pairs only, and `retarget_search_problem` writes the matching distances only for those pairs. This is both a modelling statement - lane 1 and lane 5 are never near each other, so checking them is meaningless - and a performance measure, since undeclared pairs are never grounded into actions at all.

The geometry underwrites the whole scheme. Because `_equal_area_splits` produces lanes that are disjoint by construction, and `MIN_LANE_WIDTH_M` keeps each lane at least 20 m wide against a 10 m minimum separation, corresponding waypoints in neighbouring lanes are genuinely far enough apart for the checks to pass.

5.3 The mission-wide synchronisation point

The rendezvous

```
drone1 covers lane1  -->  (area-covered forest-lane1)
drone2 covers lane2  -->  (area-covered forest-lane2)
                                  |
                      complete-forest-search
                                  |
                                  v
                       (forest-search-completed)
                                  |
            +---------------------+---------------------+
            |                     |                     |
     return-to-base          return-to-base          return-to-base
       (drone1)                (drone2)                (droneN)
```

`forest-search-completed` is a nullary fact - it describes the mission, not any drone - and it gates `return-to-base` for **everyone**. No drone may go home until the search as a whole is declared finished. This is a broadcast-style coupling: one fact, produced once, that every drone's remaining schedule depends on. It is the only place in the domain where a drone's progress is blocked by something another drone must do.

5.4 At runtime: a star through the GCS

The runtime transport

```
                    GROUND CONTROL STATION
                      (engine/gcs.py)
                  one UDP socket, port 14690
                              |
       +----------------------+----------------------+
       |                      |                      |
   udp:14701              udp:14702             udp:1470N
   Drone SYSID 1          Drone SYSID 2         Drone SYSID N

GCS  ------ COMMAND (JSON/UDP) ------>  drone
GCS  <-- TELEMETRY / HEARTBEAT / ACK --  drone
drone  <--------- X ---------->  drone      (no such link exists)
```

The separation protocol above is a **planning-time** construct. When the mission is actually flown there is no drone-to-drone radio link anywhere in the codebase: each drone binds its own UDP port at `14700 + sysid`, the GCS holds one socket that addresses the whole swarm, and every message carries `drone_id` so a single socket can demultiplex the fleet.

So the honest description for a write-up is this: the drones do not negotiate separation with each other in flight. Separation is **designed in** by the geometry - disjoint equal-area lanes at least 20 m wide - and **verified** by the planner before launch. At runtime the GCS holds the authoritative picture of every drone's position from its telemetry, sent twice a second, with heartbeats every second and a three-second timeout marking a drone as lost.

6. A worked run

The opening of a real 3-drone Search run from `plans/Search/runs/`:

ENHSP output - 3 drones, 3 lanes

```
Problem Solved
0.0:  (check-battery drone1 base drone2-wp1)
1.0:  (check-battery drone2 base drone2-wp1)
2.0:  (check-battery drone3 base drone2-wp1)
3.0:  (move-search drone1 base drone1-wp1)
4.0:  (start-search drone1 forest-lane1 drone1-wp1)
5.0:  (cover-area drone1 forest-lane1 drone1-wp1)
6.0:  (check-battery drone1 drone1-wp1 drone1-wp2)
7.0:  (check-drone-separation drone2 drone1 base drone1-wp1)
8.0:  (move-search drone2 base drone2-wp1)
9.0:  (check-drone-separation drone3 drone2 base drone2-wp1)
10.0: (check-battery drone2 drone2-wp1 drone2-wp2)
11.0: (start-search drone2 forest-lane2 drone2-wp1)
12.0: (cover-area drone2 forest-lane2 drone2-wp1)
13.0: (complete-forest-search forest-lane1 forest-lane2)
14.0: (move-search drone1 drone1-wp1 drone1-wp2)
15.0: (check-battery drone1 drone1-wp2 drone1-wp3)
16.0: (move-search drone2 drone2-wp1 drone2-wp2)
17.0: (check-drone-separation drone1 drone2 drone1-wp2 drone2-wp2)
...
```

Five things are worth reading out of this trace.

Step 3 needs no separation check. drone1 moves straight off the base because the template asserted `(separation-ok drone1)` in the initial state - the bootstrap from section 3. Every later move is preceded by a fresh check.

The chain is visible at steps 7 and 9. `check-drone-separation drone2 drone1` clears both drone2 and drone1; then `check-drone-separation drone3 drone2` clears drone3 and drone2 again. Two checks, three drones cleared - the N-1 pattern from section 5.2 in action.

Coverage is one action. Steps 4 and 5 have drone1 start and complete its entire lane at its first waypoint. The real sweep of that lane is flown from `fine_routes`; the planner treats it as atomic.

`complete-forest-search` fires at step 13, after only two lanes. This is the two-area limitation of section 2.4 visible in a real plan - lane 3 has not been covered yet. The goal's appended `(area-covered forest-lane3)` conjunct, written by `expand_search_drones`, is what still forces drone3 to do its work before the plan can end.

The three opening battery checks all name `drone2-wp1`. `check-battery` requires only `connected`, and the base is connected to the first waypoint of every lane, so the planner is free to certify a drone's battery against any of them. Since the flag is per-drone rather than per-leg, it does not matter which leg was used - `move-search` re-tests the battery for the leg it actually flies.

6.1 Record to date

Metric	Value
Runs logged	69
Solved	69
Unsolvable / no plan	0
Open code issues	None

Search is the most exercised and most reliable of the four plans, with no failures on record. Two design choices are chiefly responsible: the battery rescaling in section 4.3, which removes the commonest cause of a spurious 'unsolvable', and the coarsening in section 4.4, which keeps the planning problem small enough to always finish.

7. Summary

- **Each drone flies its own route** - the only plan of the four where that is true, and the reason `per_drone_waypoints` exists on `PlanRunResult`.
- **The waypoint graph is generated, not relocated.** The template carries no waypoints at all; every location and route fact is injected fresh on each run.
- **The coverage geometry is exact.** Convex hull, then true minimum-area bounding rectangle by rotating calipers, then an equal-area lane split by binary search over polygon clipping, then boustrophedon sweeps clipped to the real boundary at every pass.
- **Lanes are split by area, not width**, so a lopsided shape still gives every drone a fair share of ground.
- **ENHSP sees a five-point skeleton per lane, not the real sweep.** It is a feasibility check, not the source of the flight path - the drones fly `fine_routes`.
- **Separation is the drone-to-drone mechanism**, re-established before every single hop, declared only between adjacent lanes, and propagated along a chain because each check clears both of its drones.
- *`forest-search-completed` is the rendezvous* - a mission-wide fact that gates every drone's trip home.
- **At runtime there is still no peer link.** Separation is designed into the geometry and verified before launch; the transport remains a star through the GCS.